Internship Technical Report

Development and Evaluation of Chat3GPP-Based Retrieval-Augmented Question Answering Systems for 3GPP Documents

Chat3GPP and Chat3GPP-2.0

RAG Pipeline, Experiments, Results, Code Structure, and Execution Guide

Submitted by: Maneswar Sahu

Department/Lab: Wisdom Lab

Project Supervisor: Dr. Rohit Budhiraja

Mentor: Mr. Chethan. R

Academic Institution: IIT Kanpur

Primary Project Area: LLM-Assisted Retrieval-Augmented Generation for 3GPP Standards Documents

Acknowledgement

I would like to express my sincere gratitude to my project supervisor, **Dr. Rohit Budhiraja**, for providing me this internship opportunity. I would also like to thank my mentor, **Mr. Chethan R.**, for his guidance and support throughout this internship. I am thankful for the opportunity to work on Chat3GPP-based retrieval-augmented question answering systems for 3GPP telecommunications documents and for the support provided during implementation, evaluation, debugging, and report preparation.

I also acknowledge the open-source tools, models, datasets, and documentation used during this work, including Elasticsearch, Neo4j, Pathway, BGE-M3, Llama-3-8B-Instruct, Llama-3-8B-Tele-it, Mixtral-based judging, TeleQnA, Tele-Eval, and the reference Chat3GPP methodology. These resources helped in reproducing, extending, and evaluating standards-based telecom question-answering pipelines.

Abstract

This internship report documents the complete reproduced and extended Chat3GPP Retrieval-Augmented Generation pipeline for 3GPP standards-based question answering and the Chat3GPP-2.0 graph-guided RAG design. The Chat3GPP results are reported separately from the original paper results because the original paper indexed raw 3GPP Release 17/18 Word documents, while the local reproduction used processed and cleaned text content. The Chat3GPP work focuses on preprocessing, Release 17/18 filtering, Elasticsearch indexing, BGE-M3 embeddings, hybrid BM25 and dense retrieval, Reciprocal Rank Fusion, BGE-M3 ColBERT-style reranking, Llama-3-8B-Instruct generation, Llama-3-8B-Tele-it ablations, TeleQnA MCQ evaluation, Tele-Eval open-ended evaluation, and local quantized Mixtral judging. The strongest verified MCQ configuration is Llama-3-8B-Instruct with the frozen Release 17/18 RAG index, achieving 77.939% accuracy with 1180 correct answers out of 1514. The strongest verified local open-ended Tele-Eval configuration under the quantized Mixtral judge is Llama-3-8B-Tele-it with final RAG, achieving 87.489% accuracy with 51292 accepted answers out of 58627. The report also describes Chat3GPP-2.0, which combines Neo4j graph-guided document selection, Pathway streaming ingestion, hybrid retrieval, reranking, and answer generation, and proposes a fully local alternative using BGE-M3, Elasticsearch, Llama-3-8B-Instruct, and Llama-3-8B-Tele-it.

Key Results and Report Scope

Final Chat3GPP MCQ System	Llama-3-8B-Instruct + frozen Release 17/18 RAG index. Accuracy: **77.939%**; Correct: **1180 / 1514**.
Paper-Local MCQ Comparison	Original paper Chat3GPP result: **78.7%**. Local reproduced result: **77.939%**. The small difference is mainly attributed to raw Word-document indexing in the paper versus processed/cleaned text indexing locally.
Final Open-ended System	Llama-3-8B-Tele-it + final RAG under local quantized Mixtral judge. Accuracy: **87.489%**; Accepted: **51292 / 58627**.
Main Chat3GPP Finding	Retrieval quality, release filtering, hybrid BM25+dense retrieval, RRF, and reranking are more important than telecom fine-tuning alone for exact MCQ-style standards QA.
Main Chat3GPP-2.0 Finding	Neo4j graph-guided retrieval and Pathway ingestion can focus the document space before passage-level RAG; paid Voyage/API components can be replaced with local BGE-M3, Elasticsearch, Llama-3-8B-Instruct, and Tele-it.

Contents

Acknowledgement i

Abstract ii

1 Introduction 1

2 Objectives 1

3 Report Coverage Map 1

4 Background and Research Motivation 1

5 Project Folder Structure and Codebase Overview 2
- 5.1 Main Project Folders . . . 2
- 5.2 CHAT3GPP SERVER BACKUP FRESH . . . 2
- 5.3 CHAT3GPP Final Frozen Full . . . 2
- 5.4 Chat3GPP-2.0 . . . 2

6 Implementation Files and Their Roles 2
- 6.1 Preprocessing Files . . . 2
- 6.2 Index Construction Files . . . 3

6.3 Retrieval Files . . . 3
6.4 LLM and Generation Files . . . 3
6.5 Mixtral Judging Files . . . 4
6.6 Dataset Preparation Files . . . 4
6.7 Audit and Status Files . . . 4

7 Final Datasets **4**
7.1 TeleQnA Dataset . . . 4
7.2 Tele-Eval Dataset . . . 4

8 Chat3GPP Methodology and Architecture **4**

9 Preprocessing Pipeline **5**
9.1 Paper Preprocessing . . . 5
9.2 Local Preprocessing . . . 5
9.3 Removed Content . . . 5
9.4 Formatting Cleanup . . . 6
9.5 Hierarchical Structure . . . 6

10 Corpus Versions **6**
10.1 Original Corpus . . . 6
10.2 Clean Corpus . . . 6
10.3 Release 17/18 Corpus . . . 6
10.4 Final Normalization . . . 6
10.5 Final V2 Normalization . . . 6

11 Index Statistics **6**

12 Chunking Strategy **6**

13 Embeddings **7**

14 Elasticsearch Indexing **7**

15 Retrieval Pipeline **7**
15.1 BM25 Retrieval . . . 7
15.2 Dense Retrieval . . . 7
15.3 Dense Retrieval Implementation Difference . . . 7
15.4 Reciprocal Rank Fusion . . . 7
15.5 BGE-M3 ColBERT Reranking . . . 7

16 Model Configuration **7**
16.1 Llama-3-8B-Instruct . . . 7
16.2 Llama-3-8B-Tele-it . . . 7
16.3 Mixtral-8x7B-Instruct Q4 . . . 7

17 Prompting and Generation **8**
17.1 MCQ Retrieval Protocol . . . 8
17.2 MCQ Scoring . . . 8
17.3 Open-Ended Generation . . . 8

18 TeleQnA MCQ Experiments **8**
18.1 Corpus and Retrieval Evolution . . . 8
19.1 Final MCQ Result Summary . . . 8
19.2 MCQ Conclusion . . . 8

20 Tele-Eval Open-Ended Experiments **8**
20.1 Final Local Tele-Eval Results . . . 9
20.2 Tele-Eval Conclusion . . . 9

21 Paper Results for Reference **9**
21.1 Paper TeleQnA Results . . . 9
21.2 Paper Tele-Eval Results . . . 9
21.3 Original Paper vs Local Reproduction Clarification . . . 9
21.4 Tele-it + RAG as an Added Internship Extension . . . 9

22 Paper vs Local Implementation Differences **10**

23 Why the Final Configuration Is Best **10**

24 Final File Manifest **10**
24.1 Final Datasets . . . 10
24.2 Final Corpus . . . 10
24.3 Final Elasticsearch Index . . . 10
24.4 Final TeleQnA Outputs . . . 10
24.5 Final Tele-Eval Generated Outputs . . . 10
24.6 Final Mixtral Judged Outputs . . . 11
24.7 Final Scripts . . . 11

25 Server Setup and Reproduction Guide **12**
25.1 Use the GPU Server . . . 12
25.2 Activate Environment . . . 12
25.3 Create a New Environment . . . 12
25.4 Configure Model Paths . . . 12
25.5 Start and Check Elasticsearch . . . 12
25.6 Check Final Corpus . . . 12

26 Build the Final Index **12**
26.1 Preferred Resume-Based Build . . . 12
26.2 Full Rebuild Command . . . 13
26.3 Verify Index Count . . . 13

27 Run Final Chat3GPP TeleQnA **13**

28 Run Tele-it TeleQnA With RAG **14**

29 Run Tele-it TeleQnA Without RAG **14**

30 Run Tele-it TeleQnA No-RAG Paper-Faithful Variant **15**

31 Run Chat3GPP Tele-Eval Generation **15**

32 Run Tele-it Tele-Eval With RAG **15**

33 Run Tele-it Tele-Eval Without RAG **16**

34 Run Mixtral Judging **16**
34.1 Judge Chat3GPP Tele-Eval . . . 16
34.2 Judge Tele-it No-RAG . . . 16
34.3 Judge Tele-it With RAG . . . 16

35 Monitor Progress **16**

36 Compute All TeleQnA Accuracies **17**

37 Resume Behavior **17**

38 Main Research Findings **18**
38.1 MCQ Finding . . . 18
38.2 Open-Ended Finding . . . 18

38.3 Main Insight 18

39 Research Gap **18**

40 Contributions **18**

41 Known Implementation Limitations **18**

42 Future Work **19**

43 Chat3GPP Result Interpretation **19**

44 Chat3GPP-2.0 Overview **20**

45 Telecom-Data Ingestion into Neo4j **20**

46 Graph-Guided Retrieval **20**

47 Live Document Ingestion **20**

48 Current Pathway Processing **20**

49 Current Paid Configuration **21**

50 Proposed Free Local Alternative **21**

51 Proposed Alternative Retrieval Pipeline **21**

52 BGE-M3 Embeddings in Chat3GPP-2.0 **22**

53 Elasticsearch Inverted Index **22**

54 HNSW Vector Index **22**

55 Hybrid Retrieval and RRF **22**

56 BGE-M3 Reranking **22**

57 Local Generation Models **22**
57.1 MCQ Questions 22
57.2 Open-Ended Questions 23

58 Recommended Final Chat3GPP-2.0 Configuration **23**

59 Chat3GPP-2.0 Result Interpretation **23**

60 Conclusion **23**

Bibliography **24**

List of Tables

1 Coverage map for the final report. . . . 1
2 Elasticsearch index statistics. . . . 6
3 TeleQnA corpus and retrieval evolution. . . . 8
4 Final TeleQnA model comparison. . . . 8
5 Final MCQ result summary. . . . 8
6 Tele-Eval dataset split. . . . 9
7 Final local Tele-Eval results. . . . 9
8 Paper TeleQnA results for reference. . . . 9
9 Paper Tele-Eval results for reference. . . . 9
10 Original paper result versus local reproduced Chat3GPP result. . . . 9
11 Tele-it + RAG extension added in this internship. . . . 10
12 Chat3GPP-2.0 service layout. . . . 20
13 Current paid Chat3GPP-2.0 configuration. . . . 21
14 Proposed free local configuration. . . . 21
15 Recommended final Chat3GPP-2.0 local configuration. . . . 23

List of Figures

1 Chat3GPP RAG pipeline. 5
2 Chat3GPP-2.0 architecture. 21
3 Proposed free architecture for Chat3GPP-2.0. 21

1. Introduction

This report documents the complete reproduced and extended Chat3GPP Retrieval-Augmented Generation pipeline for 3GPP standards-based question answering.

The examined project contains three main folders:

- `CHAT3GPPFinalFrozenFull`
- `CHAT3GPPSERVERBACKUPFRESH`
- `Chat3GPP-2.0`

The main analysis focuses on:

- `CHAT3GPPSERVERBACKUPFRESH/Chat3GPP_fresh`
- `CHAT3GPPFinalFrozenFull`

`CHAT3GPPSERVERBACKUPFRESH/Chat3GPP_fresh` contains the experimental history, including preprocessing variants, release-filtered corpora, Elasticsearch index versions, retrieval experiments, TeleQnA and Tele-Eval scripts, no-RAG baselines, RAG ablations, corrected scoring files, Mixtral judging scripts, logs, and archived outputs.

`CHAT3GPPFinalFrozenFull` contains the selected final version, including the frozen corpus, final implementation, completed evaluation outputs, ablation files, logs, model snapshots, retrieval configuration, and quantized Mixtral judge.

The strongest verified multiple-choice configuration is:

Strongest Verified MCQ Configuration

Llama-3-8B-Instruct + frozen Release 17/18 RAG index
Accuracy: 77.939%
Correct: 1180 / 1514

The strongest verified open-ended Tele-Eval configuration under the local quantized Mixtral judge is:

Strongest Verified Open-ended Configuration

Llama-3-8B-Tele-it + final RAG
Accuracy: 87.489%
Accepted: 51292 / 58627

The final conclusion is:

- MCQ best system: **Llama-3-8B-Instruct + RAG**
- Open-ended Tele-Eval best system: **Llama-3-8B-Tele-it + RAG**

Therefore, retrieval is the dominant factor for exact standards-based 3GPP question answering, while telecom fine-tuning becomes especially beneficial for open-ended generation when combined with strong retrieved evidence.

The report is organized in the standard internship-report order: introduction, objectives, background, codebase structure, datasets, methodology, implementation details, experimental results, execution guide, Chat3GPP-2.0 design, contributions, limitations, future work, conclusion, and bibliography.

2. Objectives

The objective of this project was to reproduce and extend the Chat3GPP RAG pipeline for answering technical questions from 3GPP Release 17 and Release 18 standards.

The work involved:

1. Rebuilding and cleaning the 3GPP document corpus.
2. Filtering the corpus to Release 17 and Release 18.
3. Creating multiple Elasticsearch indexes.
4. Testing different preprocessing and normalization versions.
5. Using BGE-M3 embeddings for dense retrieval.
6. Combining BM25 and dense retrieval.
7. Applying Reciprocal Rank Fusion.
8. Applying BGE-M3 ColBERT-style reranking.
9. Generating answers using Llama-3-8B-Instruct and Llama-3-8B-Tele-it.
10. Evaluating MCQ answers using exact option matching.
11. Evaluating open-ended answers using local quantized Mixtral-8x7B-Instruct.
12. Comparing RAG and no-RAG systems.
13. Comparing general Llama with telecom-fine-tuned Tele-it.
14. Preserving the best version as a frozen final implementation.

3. Report Coverage Map

To keep the report complete while improving readability, the technical material is grouped into the following official sections.

Table 1: Coverage map for the final report.

Coverage Area	Where It Is Covered
Chat3GPP objective, motivation, folders, and files	Sections 1-5
Datasets, preprocessing, chunking, embeddings, indexing, and retrieval	Sections 6-14
Model configuration, prompting, scoring, and judging	Sections 15-16
TeleQnA, Tele-Eval, paper comparison, and final results	Sections 17-20
Execution commands, monitoring, accuracy scripts, and resume behavior	Sections 24-34
Chat3GPP-2.0 graph-guided RAG and local alternative	Sections 35-47
Contributions, limitations, future work, conclusion, and bibliography	Final sections

4. Background and Research Motivation

3GPP standards are difficult for conventional LLMs because they are:

1. Very large.

2. Highly technical.
3. Divided across thousands of documents.
4. Frequently updated.
5. Release-dependent.
6. Filled with acronyms, cross-references, and normative terminology.
7. Dependent on exact messages, interfaces, procedures, and parameter values.
8. Partly represented through tables, diagrams, and message flows.

A telecom fine-tuned model may understand telecommunications terminology better than a general model, but it cannot reliably memorize every clause from every 3GPP specification.
Fine-tuning alone has several limitations:

1. High training cost.
2. Limited telecom training data.
3. Knowledge becomes stale when specifications change.
4. Individual facts are difficult to update.
5. Source traceability is limited.
6. The model may hallucinate plausible but incorrect protocol details.

Chat3GPP addresses this by keeping standards knowledge in an external retrieval index. The model receives relevant specification passages at inference time.
The central research question is:

Can high-quality retrieval and corpus engineering allow a general 8B model to outperform a telecom-fine-tuned 8B model on exact 3GPP questions?

The MCQ results show that it can.

5. Project Folder Structure and Codebase Overview

5.1 Main Project Folders

- `CHAT3GPPFinalFrozenFull`
- `CHAT3GPPSERVERBACKUPFRESH`
- `Chat3GPP-2.0`

5.2 CHAT3GPP SERVER BACKUP FRESH

This folder contains the development and experimental repository. It includes:

1. Multiple preprocessing scripts.
2. Corpus-cleaning variants.
3. Release filtering variants.
4. Elasticsearch indexing scripts.
5. Resume indexing scripts.
6. Different Elasticsearch index versions.
7. Different TeleQnA evaluation scripts.
8. Different Tele-Eval evaluation scripts.
9. Tele-it RAG scripts.
10. Tele-it no-RAG scripts.
11. Mixtral judging scripts.
12. Intermediate JSON outputs.
13. Failed or partially completed logs.
14. Archived old results.
15. Audit files and experiment summaries.

This folder is useful for understanding how the final version was reached. It should not be treated as the final reproducible folder unless the final selected files are clearly identified.

5.3 CHAT3GPP Final Frozen Full

This folder preserves the final selected implementation. It contains:

1. The selected 1843-file Release 17/18 final corpus.
2. The final retrieval pipeline.
3. The final generation scripts.
4. The final TeleQnA outputs.
5. The final Tele-Eval outputs.
6. The final Mixtral evaluation outputs.
7. Tele-it ablation outputs.
8. Paper-faithful Q4 Mixtral judging outputs.
9. Llama, Tele-it, BGE-M3, and Mixtral model snapshots.
10. Frozen logs.
11. Final retrieval and model configuration.

This folder should be treated as the authoritative frozen version.

5.4 Chat3GPP-2.0

`Chat3GPP-2.0` is handled as a separate major part of this report because its architecture differs from the frozen Chat3GPP reproduction. The verified reproduced Chat3GPP numerical results are reported from `CHAT3GPPFinalFrozenFull` and `CHAT3GPPSERVERBACKUPFRESH/Chat3GPP_fresh`, while Chat3GPP-2.0 is documented separately for its Neo4j graph-guided retrieval, Pathway ingestion, paid-service configuration, and proposed free local alternative.

6. Implementation Files and Their Roles

6.1 Preprocessing Files

- `build_clean_corpus.py`
- `normalize_rel1718_final.py`
- `normalize_rel1718_final_v2.py`
- `preprocessing_comparison.txt`
- `release_map.json`

Roles:

1. build_clean_corpus.py: Builds the cleaned text corpus from extracted 3GPP text.
2. normalize_rel1718_final.py: Performs the selected final normalization for the Release 17/18 corpus.
3. normalize_rel1718_final_v2.py: Performs the later v2 normalization experiment, which preserved additional bracketed references but reduced MCQ accuracy.
4. preprocessing_comparison.txt: Stores comparison notes between preprocessing variants.
5. release_map.json: Maps documents to 3GPP releases and is used for Release 17/18 filtering.

6.2 Index Construction Files

- init_database_clean.py
- init_database_clean_rel1718.py
- init_database_clean_rel1718_final.py
- index_one_clean_file.py
- index_one_clean_rel1718_file.py
- index_one_clean_rel1718_final_file.py
- index_one_clean_rel1718_final_v2_file.py
- resume_clean_index_exact.py
- resume_clean_index_rel1718_exact.py
- resume_clean_index_rel1718_final_exact.py
- resume_clean_index_rel1718_final_v2_exact.py

Roles:

1. init_database_clean.py: Builds the cleaned full-corpus Elasticsearch index.
2. init_database_clean_rel1718.py: Builds the Release 17/18 cleaned index. In the frozen folder, this may target index_clean_rel1718, not necessarily the final index.
3. init_database_clean_rel1718_final.py: Builds the selected final Release 17/18 index when available.
4. index_one_clean_file.py: Indexes one file into the clean corpus index.
5. index_one_clean_rel1718_file.py: Indexes one file into the Release 17/18 clean index.
6. index_one_clean_rel1718_final_file.py: Indexes one file into the selected final Release 17/18 index.
7. index_one_clean_rel1718_final_v2_file.py: Indexes one file into the final v2 Release 17/18 index.
8. resume_clean_index_exact.py: Resumes interrupted indexing for the clean index.
9. resume_clean_index_rel1718_exact.py: Resumes interrupted indexing for the Release 17/18 clean index.
10. resume_clean_index_rel1718_final_exact.py: Resumes interrupted indexing for the selected final Release 17/18 index.
11. resume_clean_index_rel1718_final_v2_exact.py: Resumes interrupted indexing for the final v2 Release 17/18 index.

Implementation note: The frozen init_database_clean_rel1718.py may target index_clean_rel1718, not index_clean_rel1718_final. For rebuilding the exact final selected index, use the final resume/index scripts from the backup folder or correct the index name and corpus path.

6.3 Retrieval Files

- retrievers/VectorRetriever.py
- retrievers/base.py
- server/embedding.py
- server/reranker.py

Roles:

1. retrievers/VectorRetriever.py: Implements the main retrieval pipeline, including Elasticsearch access, BM25 retrieval, dense vector retrieval, Reciprocal Rank Fusion, candidate filtering, reranking, and final top-k retrieval.
2. retrievers/base.py: Defines the base retrieval interface or shared retrieval classes.
3. server/embedding.py: Loads BGE-M3 and provides query/-document embedding functions.
4. server/reranker.py: Loads BGE-M3 reranking or ColBERT-style late-interaction scoring logic.

6.4 LLM and Generation Files

- server/llm.py
- run_teleqna_eval_clean.py
- run_teleqna_eval_clean_rel1718.py
- run_teleqna_eval_clean_rel1718_final.py
- run_teleqna_eval_clean_rel1718_final_v2.py
- run_teleeval_eval_clean_rel1718_final.py
- run_tele_it_teleqna_clean_rel1718_final_gpu0.py
- run_tele_it_teleeval_clean_rel1718_final.py
- run_tele_it_teleqna_no_rag.py
- run_tele_it_teleqna_no_rag_paperfaithful.py
- run_teleit_teleeval_no_rag.py

Roles:

1. server/llm.py: Loads the LLM and tokenizer, applies deterministic generation, sets padding behavior, and returns decoded model outputs.
2. run_teleqna_eval_clean.py: Runs TeleQnA on the broader clean index.
3. run_teleqna_eval_clean_rel1718.py: Runs TeleQnA on the Release 17/18 clean index.
4. run_teleqna_eval_clean_rel1718_final.py: Runs the final selected Chat3GPP MCQ evaluation.
5. run_teleqna_eval_clean_rel1718_final_v2.py: Runs the v2 normalization MCQ experiment.
6. run_teleeval_eval_clean_rel1718_final.py: Runs Chat3GPP open-ended Tele-Eval generation on the final index.

7. `run_tele_it_teleqna_clean_rel1718_final_gpu0.py`: Runs Tele-it with the final RAG pipeline for TeleQnA.
8. `run_tele_it_teleeval_clean_rel1718_final.py`: Runs Tele-it with the final RAG pipeline for Tele-Eval.
9. `run_tele_it_teleqna_no_rag.py`: Runs Tele-it MCQ without RAG.
10. `run_tele_it_teleqna_no_rag_paperfaithful.py`: Runs Tele-it MCQ without RAG using a paper-faithful prompt.
11. `run_teleit_teleeval_no_rag.py`: Runs Tele-it open-ended Tele-Eval without RAG.

6.5 Mixtral Judging Files

- `run_mixtral_teleeval_judge_paperfaithful.py`
- `run_mixtral_teleeval_judge_paperfaithful_v2.py`
- `run_mixtral_teleeval_judge_paperfaithful_v3.py`

Roles:

1. `run_mixtral_teleeval_judge_paperfaithful.py`: Initial local Mixtral judging script.
2. `run_mixtral_teleeval_judge_paperfaithful_v2.py`: Updated judge version.
3. `run_mixtral_teleeval_judge_paperfaithful_v3.py`: Final paper-faithful Q4 Mixtral judging script used for final results.

The final selected judging script is `run_mixtral_teleeval_judge_paperfaithful_v3.py`.

6.6 Dataset Preparation Files

- `prepare_teleqna_paper.py`
- `build_release_map.py`
- `filter_teleeval_58k.py`

Roles:

1. `prepare_teleqna_paper.py`: Prepares the final TeleQnA paper subset.
2. `build_release_map.py`: Builds release mapping information for filtering 3GPP documents.
3. `filter_teleeval_58k.py`: Produces the final 58627-question Tele-Eval subset.

6.7 Audit and Status Files

- `experiment_status_now.txt`
- `teleqna_final_scores.txt`
- `teleqna_completed_runs.txt`
- `audit_full/`
- `archive_old/results/teleqna/`
- `outputs/`

Roles:

1. `experiment_status_now.txt`: Contains experiment status notes.
2. `teleqna_final_scores.txt`: Stores final TeleQnA scoring information.
3. `teleqna_completed_runs.txt`: Lists completed TeleQnA runs.
4. `audit_full/`: Contains audit material and verification files.
5. `archive_old/results/teleqna/`: Contains older TeleQnA result files.
6. `outputs/`: Contains final and intermediate output JSON files.

7. Final Datasets

7.1 TeleQnA Dataset

Final file: `data/teleqna/teleqna_paper.json`

- Release 17: 734
- Release 18: 780
- Total: 1514
- Evaluation type: multiple-choice exact option matching

7.2 Tele-Eval Dataset

Final file: `data/teleeval_58627_exact.json`

- Release 17: 26225
- Release 18: 32402
- Total: 58627
- Evaluation type: open-ended LLM-based judging

8. Chat3GPP Methodology and Architecture

The final Chat3GPP pipeline is:

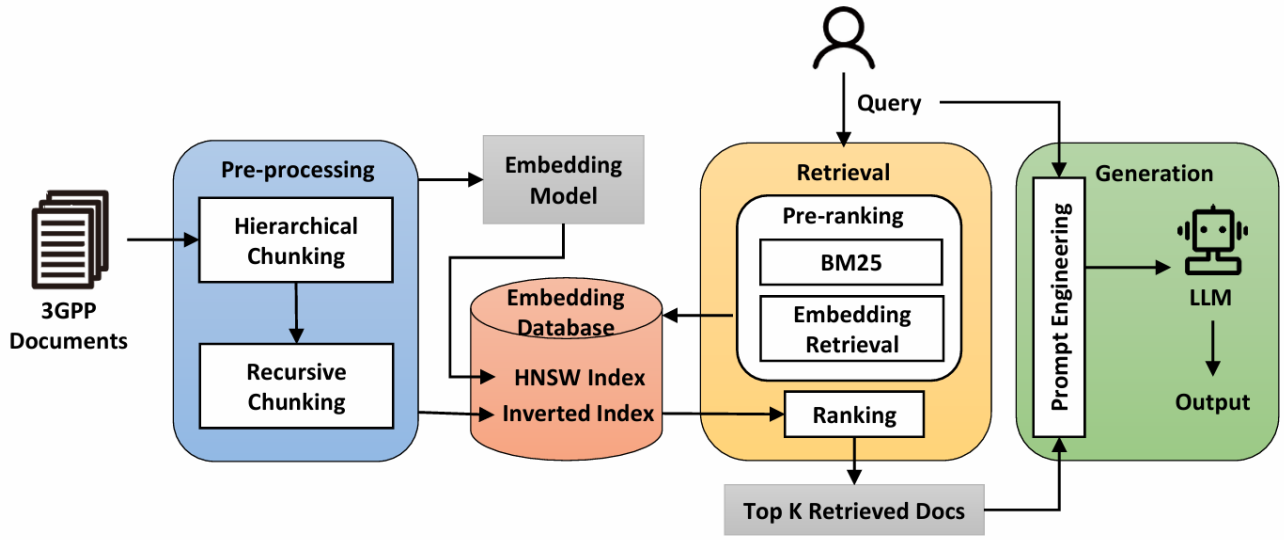


Figure 1: Chat3GPP RAG pipeline.

3GPP Release 17/18 documents
↓
`.doc` to `.docx` conversion where necessary
↓
Text extraction
↓
Contents / references / annex / boilerplate filtering
↓
Hierarchical heading reconstruction
↓
Formatting and LaTeX cleanup
↓
Recursive 1250-character chunks
↓
BGE-M3 1024-dimensional dense embeddings
↓
Elasticsearch content and vector index
↓
BM25 top 1000 candidates + Dense cosine top 1000 candidates
↓
Reciprocal Rank Fusion
↓
Top 100 fused candidates
↓
BGE-M3 ColBERT late-interaction reranking
↓
Top five evidence chunks
↓
Llama-3-8B-Instruct or Llama-3-8B-Tele-it
↓
MCQ or open-ended answer
↓
Exact option scoring or Mixtral LLM-Eval

9. Preprocessing Pipeline

9.1 Paper Preprocessing

The paper pipeline uses raw 3GPP `.doc` and `.docx` documents. The paper methodology includes:

1. Downloading Release 17 and Release 18 3GPP technical specifications.
2. Converting older `.doc` files to `.docx`.
3. Extracting text from Word documents.
4. Removing contents sections.
5. Removing references sections.
6. Removing annexes.
7. Removing images.
8. Removing tables.
9. Applying hierarchical chunking.
10. Applying recursive splitting.
11. Creating embeddings.
12. Indexing chunks into Elasticsearch.

9.2 Local Preprocessing

The local reproduction used processed text content derived from the available Tele-Data / standards corpus rather than fully reproducing the raw Word-document pipeline. Therefore, the local corpus is content-faithful, but not fully structure-identical to the paper's raw `.doc/.docx` preprocessing path. This is an important implementation difference and should be stated clearly.

9.3 Removed Content

The local preprocessing removed or skipped:

1. Forewords.
2. Contents pages.
3. Tables of contents.

4. References.
5. Annexes.
6. Version descriptions.
7. Repeated 3GPP legal boilerplate.
8. Very short or empty sections.
9. Formatting artifacts.
10. Non-answer-bearing text.

These passages often contain keywords but rarely contain the evidence required to answer questions.

9.4 Formatting Cleanup

The code normalizes:

1. Null characters.
2. Tabs.
3. Carriage returns.
4. Multiple spaces.
5. LaTeX `begin` and `end` commands.
6. `\item .`
7. LaTeX formatting commands.
8. Escaped underscores.
9. Tildes.
10. Braces.
11. Malformed enumeration markers.
12. API/path escape artifacts in later variants.

9.5 Hierarchical Structure

Numbered headings are detected and stored using a stack based on heading depth. A section is transformed into a structured chunk such as:

```
Title Path: Main heading > Subheading > Procedure
Content: The actual specification text...
```

This prevents important structural information from disappearing after chunking.

10. Corpus Versions

Several corpus and index variants were created.

10.1 Original Corpus

The original index retained substantial boilerplate and irrelevant material.

- Approximate index size: 3446573 chunks
- Approximate storage size: 4.6 GB
- MCQ accuracy: 74.108%

10.2 Clean Corpus

The first cleaned corpus introduced hierarchical heading preservation, boilerplate filtering, LaTeX cleanup, and short-section removal.

- Index: `index_clean`
- Accuracy: 76.024%

10.3 Release 17/18 Corpus

The corpus was then limited to documents mapped to Release 17 and Release 18. This reduced cross-release conflicts, out-of-scope retrieval, duplicate passages, candidate competition, and retrieval noise.

10.4 Final Normalization

- Final corpus folder: `knowledge_base_cleaned_rel1718_final/`
- Final index: `index_clean_rel1718_final`
- Document count: 1843 source files
- Chunk count: 254570 chunks
- Accuracy: 77.939%

This was the selected final configuration.

10.5 Final V2 Normalization

- Final v2 corpus folder: `knowledge_base_cleaned_rel1718_final_v2/`
- Index: `index_clean_rel1718_final_v2`
- Preserved bracketed references such as `[4]`, `[14]`, and `[A.2]`.
- Repaired certain API/path escape patterns.
- Produced lower MCQ accuracy: 77.411%.

Therefore, it was not selected. Although v2 preserved more document information, the additional reference tokens increased retrieval noise.

11. Index Statistics

Table 2: Elasticsearch index statistics.

Index	Docs/Chunks	Size
Original `index`	3446573	4.6 GB
`index_clean`	398279	5.2 GB
`index_clean_rel1718`	288389	3.7 GB
`index_clean_rel1718_final`	254570	3.4 GB
`index_clean_rel1718_final_v2`	255797	3.4 GB

Final selected corpus: `knowledge_base_cleaned_rel1718_final/`. Final selected index: `index_clean_rel1718_final`. Final source file count: 1843 files.

12. Chunking Strategy

Final chunking configuration:

```
SPLITTER_CONFIG = {
    "chunk_size": 1250,
    "chunk_overlap": 0,
}
```

The system uses `RecursiveCharacterTextSplitter`. Advantages include better preservation of natural text boundaries, less sentence fragmentation, no dependence on one tokenizer, reduced duplicated content compared with overlapping windows, and compatibility with BGE-M3 context constraints. The main limitation is that zero overlap can split supporting evidence across adjacent chunks.

13. Embeddings

Embedding model: **BGE-M3**.

```
EMBED_CONFIG = {
    "embed_model": "bge-m3",
    "embed_device": "cuda:1",
}
```

Indexing uses:

```
return_dense=True
return_sparse=False
return_colbert_vecs=False
```

Each chunk receives a 1024-dimensional dense vector. BGE-M3 is also used for ColBERT reranking, but ColBERT vectors are generated dynamically rather than stored in Elasticsearch. **Implementation note:** `server/embedding.py` hardcodes `devices="cuda:1"` in some versions. Change this if BGE-M3 must run on another GPU.

14. Elasticsearch Indexing

Each indexed record stores:

```
{
    "content": "text",
    "embedding": "embedding",
    "filename": "filename",
    "date": "date"
}
```

Mapping:

- `content`: text
- `embedding`: dense vector, 1024 dimensions, cosine similarity
- `filename`: keyword
- `date`: date

Text analyzer: standard tokenizer, lowercase filter, and ASCII folding. The code assumes Elasticsearch is available at `http://localhost:9200` without authentication.

15. Retrieval Pipeline

15.1 BM25 Retrieval

BM25 retrieves exact lexical matches. It is especially useful for 3GPP acronyms, message names, interface names, information elements, protocol identifiers, exact numerical terminology, and clause-like technical wording.

15.2 Dense Retrieval

The query is encoded using BGE-M3 and compared against stored chunk embeddings using cosine similarity. Dense retrieval handles paraphrased questions, semantically related terminology, questions that do not repeat exact specification wording, and broader conceptual matching.

15.3 Dense Retrieval Implementation Difference

The paper describes HNSW retrieval. The final local code uses Elasticsearch dense retrieval with scripted cosine scoring in the form:

```
script_score match_all cosineSimilarity(...)
```

This is architecturally similar but not bit-identical to explicit Elasticsearch `knn` HNSW querying.

15.4 Reciprocal Rank Fusion

RRF formula used:

```
1.0 / (10 + rank)
```

Scores from BM25 and dense retrieval are added for documents appearing in either list. Configuration:

- BM25 Top-K1: 1000
- Dense Top-K1: 1000
- RRF constant: 10
- Fused candidates: 100
- Final Top-K2: 5

15.5 BGE-M3 ColBERT Reranking

For each fused candidate, the query is encoded into BGE-M3 ColBERT vectors, the document is encoded into ColBERT vectors, BGE-M3 late-interaction scoring calculates relevance, candidates are reordered, and the top five chunks are returned. The paper calls the reranker a cross-encoder; the local implementation behaves as a BGE-M3 ColBERT-style late-interaction reranker.

16. Model Configuration

16.1 Llama-3-8B-Instruct

Role: main Chat3GPP generator.

```
LLM_CONFIG = {
    "llm_model": "llama3-8B-instruct",
    "llm_device": "cuda:1",
    "dtype": "fp16",
}
```

Some versions use `"llm_device": "cuda:0"` depending on GPU assignment. Generation behavior:

```
do_sample=False
max_new_tokens=256
seed=42
repetition_penalty=1.05
pad_token_id=tokenizer.eos_token_id
attention_mask=inputs["attention_mask"]
```

The generator decodes only newly generated tokens.

16.2 Llama-3-8B-Tele-it

Role: telecom-fine-tuned baseline and RAG ablation model. It is used in TeleQnA without RAG, TeleQnA with RAG, Tele-Eval without RAG, and Tele-Eval with RAG.

16.3 Mixtral-8x7B-Instruct Q4

Role: local evaluator for open-ended Tele-Eval. Model file: `mixtral-8x7b-instruct-v0.1-q4_0.gguf`. Approximate file size: 26.44 GB. Configuration:

- Runtime: llama.cpp
- Quantization: Q4_0
- GPU layers: 30
- Judge context: 1024 tokens
- Threads: 8
- Temperature: 0
- Top-p: 1

- Output grammar: Yes or No
- Checkpoint: every 100 examples

The local Q4 judge is not identical to the full-precision Mixtral model used in the paper. Quantization and the 1024-token judge context can change judge decisions.

17. Prompting and Generation

17.1 MCQ Retrieval Protocol

For MCQ experiments, only the question is used for retrieval. Options are added afterward during generation. This is important because retrieving with the answer options could unfairly match the correct answer text and leak information into the retrieved context.

17.2 MCQ Scoring

The generated response is parsed using an option-number regular expression. Examples of possible model outputs:

```
2
Answer: 2
"answer": "option 2: ..."
option 3: Uplink RAN Configuration Transfer
```

The extracted option number is compared with the gold option number. This correction was important for Tele-it RAG scoring because one archived Tele-it RAG output initially appeared as 0%, but the issue was a formatting mismatch between numeric predictions and gold option strings. Corrected Tele-it RAG score: 70.608%.

17.3 Open-Ended Generation

For Tele-Eval, the model generates a direct telecommunications answer using the retrieved context. There is no simple exact-match scoring because valid answers can be phrased differently from the reference. Therefore, a local Mixtral judge compares the question, ground-truth answer, and generated answer. The judge outputs Yes or No. Invalid, empty, or failed judge outputs are counted as incorrect.

18. TeleQnA MCQ Experiments

Dataset: Release 17 has 734 questions, Release 18 has 780 questions, and the total is 1514 questions.

18.1 Corpus and Retrieval Evolution

Table 3: TeleQnA corpus and retrieval evolution.

Experiment	Release 17	Release 18	Overall	Correct
Original / k10	72.752%	75.385%	74.108%	1122
Original / k60	72.888%	75.256%	74.108%	1122
Clean hierarchical index	75.204%	76.795%	76.024%	1151
Frozen final	77.793%	78.077%	77.939%	1180
Final v2	77.248%	77.564%	77.411%	1172

Z

Table 4: Final TeleQnA model comparison.

System	RAG	Release 17	Release 18	Overall
Llama-3-8B-Instruct + final RAG	Yes	77.793%	78.077%	77.939%
Llama-3-8B-Tele-it + final RAG	Yes	70.708%	70.513%	70.608%
Tele-it without RAG	No	53.815%	50.641%	52.180%
Tele-it without RAG, paper-faithful	No	50.272%	52.308%	51.321%

19.1 Final MCQ Result Summary

Table 5: Final MCQ result summary.

Configuration	Correct	Accuracy
Llama-3-8B-Instruct + final RAG	1180 / 1514	77.939%
Tele-it + final RAG	1069 / 1514	70.608%
Tele-it without RAG	790 / 1514	52.180%
Tele-it no-RAG, paper-faithful	777 / 1514	51.321%

19.2 MCQ Conclusion

The best MCQ system is Llama-3-8B-Instruct + final RAG with 77.939% accuracy. Compared with paper-faithful Tele-it no-RAG, the gain is 26.618 percentage points, 403 additional correct answers, approximately 51.9% relative accuracy improvement, and approximately 54.7% error reduction. Compared with Tele-it using the same RAG pipeline, the gain is 7.331 percentage points.

20. Tele-Eval Open-Ended Experiments

Dataset:

Table 6: Tele-Eval dataset split.

Release	Questions
Release 17	26225
Release 18	32402
Total	58627

Evaluation uses the local quantized Mixtral-8x7B-Instruct Q4 judge.

20.1 Final Local Tele-Eval Results

Table 7: Final local Tele-Eval results.

System	Release 17	Release 18	Overall	Accepted
Llama-3-8B-Instruct + RAG	73.697%	71.959%	72.736%	42643
Tele-it without RAG	25.441%	25.137%	25.273%	14817
Tele-it + RAG	86.982%	87.899%	87.489%	51292

20.2 Tele-Eval Conclusion

The best local open-ended Tele-Eval system is Llama-3-8B-Tele-it + RAG with 87.489% accuracy. RAG improves Tele-it by 62.216 percentage points, from 25.273% to 87.489%. This shows that retrieval is essential for Tele-it, especially in open-ended QA.

21. Paper Results for Reference

Paper results and local results must be reported separately because the local pipeline used a quantized Mixtral judge and had implementation differences.

21.1 Paper TeleQnA Results

Table 8: Paper TeleQnA results for reference.

Model	Rel. 17	Rel. 18	Overall
TelecomGPT	–	–	67.1%
Llama-3-8B-Tele-it	53.1%	57.1%	55.2%
Telco-RAG	72.5%	78.4%	–
Chat3GPP	78.3%	79.1%	78.7%

21.2 Paper Tele-Eval Results

Table 9: Paper Tele-Eval results for reference.

Model	Rel. 17	Rel. 18	Overall
Llama-3-8B-Tele-it	28.3%	26.5%	27.3%
Chat3GPP	50.6%	57.3%	54.3%

21.3 Original Paper vs Local Reproduction Clarification

The original Chat3GPP paper and this internship reproduction should be interpreted as closely related but not bit-identical experiments. The original paper crawled Release 17 and Release 18 technical specification documents from the 3GPP FTP site, converted older `.doc` files into `.docx`, filtered sections such as Contents, References, and Annexes, extracted text while excluding images and tables, and then built the retrieval database using hierarchical chunking, recursive chunking, BGE-M3 embeddings, Elasticsearch inverted indexing, HNSW vector indexing, BM25, embedding retrieval, RRF, reranking, and Llama-3-8B-Instruct generation [1].

In the local implementation, the final index was built from processed and cleaned text content derived from the available Tele-Data / standards corpus rather than by fully repeating the raw `.doc/.docx` pipeline. Therefore, the local corpus is content-faithful but not structure-identical to the original paper's raw-document preprocessing path. This is the main reason a small difference exists between the paper's Chat3GPP TeleQnA result and the reproduced local result.

Table 10: Original paper result versus local reproduced Chat3GPP result.

Configuration	Rel. 17	Rel. 18	Overall
Original paper Chat3GPP	78.3%	79.1%	78.7%
Local reproduced Chat3GPP final	77.793%	78.077%	77.939%
Difference	-0.507 pp	-1.023 pp	-0.761 pp

Indexing note: The original paper used raw Release 17/18 `.doc/.docx` documents that were crawled, converted, cleaned, chunked, embedded, and indexed. The local reproduced final result used a processed/cleaned text corpus and the final index `index_clean_rel1718_final`. Therefore, the small difference is expected due to non-identical preprocessing and indexing sources.

The paper-local TeleQnA difference is therefore not a contradiction. The reproduced result remains close to the original paper result, but an exact match is not expected because the document source, preprocessing path, chunk boundaries, and some local retrieval details differ from the paper pipeline.

21.4 Tele-it + RAG as an Added Internship Extension

The original Chat3GPP paper reported Chat3GPP using Llama-3-8B-Instruct with RAG and used Llama-3-8B-Tele-it as a telecom-fine-tuned benchmark/baseline. It did not report a separate experiment in which the Tele-it model was connected to the same Chat3GPP retrieval pipeline as a dedicated Tele-it + RAG branch. This internship added that missing ablation by implementing and evaluating Tele-it with the final Release 17/18 RAG pipeline.

Table 11: Tele-it + RAG extension added in this internship.

Item	Original Paper	This Internship
Tele-it without RAG	Reported as benchmark/fine-tuned model	Reproduced and evaluated locally
Tele-it + Chat3GPP RAG	Not separately reported	Implemented and evaluated as an additional branch
MCQ result	Not reported for Tele-it + RAG	70.608%
Open-ended result	Not reported for Tele-it + RAG	87.489% under local Q4 Mixtral judge

This added branch is important because it directly measures how much retrieval improves the telecom-fine-tuned Tele-it model. The results show that retrieval substantially improves Tele-it: from 51.321% paper-faithful no-RAG MCQ to 70.608% with RAG, and from 25.273% no-RAG Tele-Eval to 87.489% with RAG under the local quantized Mixtral judge.

22. Paper vs Local Implementation Differences

- **Source documents.** Paper: raw 3GPP Release 17/18 `.doc` and `.docx` documents. Local implementation: processed text content derived from available Tele-Data / standard JSONL and cleaned text corpus. The local corpus is content-faithful but not fully structure-identical to the paper's raw Word-document preprocessing. This explains why the local Chat3GPP result, 77.939%, is close to but not exactly equal to the paper result, 78.7%.
- **Tele-it with RAG extension.** The original Chat3GPP paper reported the Chat3GPP RAG system and the Llama-3-8B-Tele-it baseline, but it did not report a separate configuration where the Tele-it model was connected to the same Chat3GPP retrieval pipeline as a dedicated Tele-it + RAG branch. In this internship work, that additional branch was implemented and evaluated. Tele-it was tested both without RAG and with the final Release 17/18 RAG pipeline, allowing a direct measurement of how much retrieval improves the telecom-fine-tuned model. This extension produced the corrected Tele-it + RAG MCQ result of 70.608% and the strongest local open-ended Tele-Eval result of 87.489%.
- **Mixtral evaluator.** Paper: full Mixtral-8x7B-Instruct. Local: Q4 quantized Mixtral-8x7B-Instruct GGUF. Full model was not feasible under local storage constraints.
- **Dense retrieval.** Paper: Elasticsearch HNSW dense vector retrieval. Local: Elasticsearch dense vector with scripted cosine similarity in some final retriever versions.
- **Reranking.** Paper description: BGE-M3 reranking / cross-encoder-style reranker. Local behavior: BGE-M3 ColBERT-style late-interaction reranking.
- **Prompting and answer parsing.** Several prompt versions were tested. The final scoring logic extracts option numbers before comparison to avoid false incorrect scores caused by formatting differences.

23. Why the Final Configuration Is Best

The final version achieves the best MCQ accuracy because it combines:

1. Release-specific corpus restriction.
2. Hierarchical heading preservation.
3. Removal of irrelevant boilerplate.
4. Conservative formatting normalization.
5. Smaller and less competitive retrieval space.
6. BM25 exact-term matching.
7. BGE-M3 semantic retrieval.
8. Reciprocal Rank Fusion.
9. BGE-M3 token-level ColBERT reranking.
10. Top-five high-quality retrieved context passages.
11. Question-only retrieval for MCQs.
12. Deterministic answer generation.
13. Corrected answer parsing.
14. Full-dataset empirical selection.
15. Rejection of the lower-performing final v2 corpus.

The result was selected based on measured accuracy, not because the folder was simply named final.

24. Final File Manifest

24.1 Final Datasets

- `data/teleqna/teleqna_paper.json`
- `data/teleeval_58627_exact.json`
- `release_map.json`

24.2 Final Corpus

`knowledge_base_cleaned_rel1718_final/`. Expected source file count: 1843.

24.3 Final Elasticsearch Index

`index_clean_rel1718_final`. Expected chunk count: 254570.

24.4 Final TeleQnA Outputs

- `outputs/teleqna/chat3gpp_teleqna_clean_rel1718_final_77939.json`
- `outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final_fixed.json`
- `outputs/teleqna/tele_it_teleqna_no_rag.json`
- `outputs/teleqna/tele_it_teleqna_no_rag_paperfaithful.json`

24.5 Final Tele-Eval Generated Outputs

- `outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json`
- `outputs/teleeval/tele_it_teleeval_results_clean_rel1718_final.json`

Additional RAG-based Tele-it Tele-Eval variants, if present:

- outputs/teleeval/tele_it_teleeval_results_cleanindex.json
- outputs/teleeval/tele_it_teleeval_results_clean_rel1718_final_v2.json

24.6 Final Mixtral Judged Outputs

- outputs/teleeval/chat3gpp_mixtral_eval_paperfaithful_v3.json
- outputs/teleeval/tele_it_no_rag_mixtral_eval_paperfaithful_v3.json
- outputs/teleeval/tele_it_rag_mixtral_eval_paperfaithful_v3.json

24.7 Final Scripts

- init_database_clean_rel1718_final.py
- run_teleqna_eval_clean_rel1718_final.py
- run_teleeval_eval_clean_rel1718_final.py
- run_tele_it_teleqna_clean_rel1718_final_gpu0.py
- run_tele_it_teleqna_no_rag.py
- run_tele_it_teleqna_no_rag_paperfaithful.py
- run_tele_it_teleeval_clean_rel1718_final.py
- run_teleit_teleeval_no_rag.py
- run_mixtral_teleeval_judge_paperfaithful_v3.py
- prepare_teleqna_paper.py
- build_release_map.py
- filter_teleeval_58k.py

25. Server Setup and Reproduction Guide

25.1 Use the GPU Server

The scripts contain Linux paths such as `/scratch/work/yash22/`. Therefore, the Windows folder should be treated mainly as an archive. The pipeline should be run on a Linux GPU server.

25.2 Activate Environment

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate
```

Possible environments mentioned: `venv`, `venv_mixtral`, `venv_mixtral_q4`, and `venv_mixtral_eval`.

25.3 Create a New Environment

```
python3.9 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
pip install FlagEmbedding
pip install llama-cpp-python
```

Important note: `FlagEmbedding` and `llama-cpp-python` are imported by final code but may not be clearly pinned in the main requirements file.

25.4 Configure Model Paths

Edit `configs/model_configs.py` and `server/embedding.py`. Set paths similar to:

```
MODEL_PATH = {
    "embed_model": {
        "bge-m3": "/path/to/bge-m3"
    },
    "llm_model": {
        "llama3-8B-instruct": "/path/to/llama3",
        "LLama-3-8B-Tele-it": "/path/to/tele-it"
    }
}
```

Check GPU assignment:

```
EMBED_CONFIG = {
    "embed_model": "bge-m3",
    "embed_device": "cuda:1",
}

LLM_CONFIG = {
    "llm_model": "llama3-8B-instruct",
    "llm_device": "cuda:0",
    "dtype": "fp16",
}
```

If needed, change `cuda:0` and `cuda:1` according to available GPUs.

25.5 Start and Check Elasticsearch

```
curl http://localhost:9200
curl "http://localhost:9200/_cat/indices?v"
```

The code assumes `http://localhost:9200` without authentication.

25.6 Check Final Corpus

```
find knowledge_base_cleaned_rel1718_final \
  -maxdepth 1 -name "*.txt" | wc -l
```

Expected: 1843.

26. Build the Final Index

26.1 Preferred Resume-Based Build

From the experimental backup project:

```
python resume_clean_index_rel1718_final_exact.py
```

This script targets `index_clean_rel1718_final`, finds completed filenames, skips already indexed documents, runs the single-file indexer for missing documents, and supports recovery after interruption.

26.2 Full Rebuild Command

If using a clean rebuild script:

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate

curl -X DELETE "localhost:9200/index_clean_rel1718_final"

python init_database_clean_rel1718_final.py 2>&1 | tee indexing_clean_rel1718_final.log
```

Important note: If `init_database_clean_rel1718_final.py` is not present in the frozen folder, use the final resume/index scripts from the backup folder or edit `init_database_clean_rel1718.py` so that both the index name and corpus path point to the final version.

26.3 Verify Index Count

Using curl:

```
curl "http://localhost:9200/index_clean_rel1718_final/_count?pretty"
```

Expected: 254570.
Using Python:

```
python - <<'PY'
from elasticsearch import Elasticsearch

es = Elasticsearch(
    "http://localhost:9200",
    request_timeout=120,
    max_retries=3,
    retry_on_timeout=True
)

res = es.search(
    index="index_clean_rel1718_final",
    body={
        "size": 0,
        "aggs": {
            "files": {
                "terms": {
                    "field": "filename",
                    "size": 10000
                }
            }
        }
    }
)

print("Indexed files:", len(res["aggregations"]["files"]["buckets"]))
PY
```

Expected output: `Indexed files:  1843`.

27. Run Final Chat3GPP TeleQnA

Script: `run_teleqna_eval_clean_rel1718_final.py`. Dataset: `data/teleqna/teleqna_paper.json`. Index: `index_clean_rel1718_final`.

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate

rm -f outputs/teleqna/teleqna_results_clean_rel1718_final.json
rm -f teleqna_clean_rel1718_final.log

nohup python run_teleqna_eval_clean_rel1718_final.py \
  > teleqna_clean_rel1718_final.log 2>&1 &

echo "PID: $!"
```

Check progress:

```
tail -f teleqna_clean_rel1718_final.log
```

Check saved answers:

```
python - <<'PY'
import json, os

p = "outputs/teleqna/teleqna_results_clean_rel1718_final.json"
print("Exists:", os.path.exists(p))

if os.path.exists(p):
    with open(p, "r", encoding="utf-8") as f:
        d = json.load(f)
    print("Saved answers:", len(d))
```

```
PY
```

Expected final result: 1180 / 1514 and 77.939%.
Compute accuracy:

```
python - <<'PY'
import json

path = "outputs/teleqna/chat3gpp_teleqna_clean_rel1718_final_77939.json"

with open(path, "r", encoding="utf-8") as f:
    data = json.load(f)

total = len(data)
correct = sum(1 for v in data.values() if v.get("is_correct", False))
acc = 100 * correct / total if total else 0

print("Total:", total)
print("Correct:", correct)
print("Accuracy:", f"{acc:.3f}%")
PY
```

Expected output: total 1514, correct 1180, accuracy 77.939%.

28. Run Tele-it TeleQnA With RAG

Script: `run_tele_it_teleqna_clean_rel1718_final_gpu0.py`. Dataset: `data/teleqna/teleqna_paper.json`. Index: `index_clean_rel1718_final`.

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate

rm -f outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final.json
rm -f teleit_teleqna_clean_rel1718_final.log

nohup python run_tele_it_teleqna_clean_rel1718_final_gpu0.py \
  > teleit_teleqna_clean_rel1718_final.log 2>&1 &

echo "PID: $!"
```

Check progress:

```
tail -f teleit_teleqna_clean_rel1718_final.log
```

Corrected output: `outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final_fixed.json`.
Compute corrected accuracy:

```
python - <<'PY'
import json

path = "outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final_fixed.json"

with open(path, "r", encoding="utf-8") as f:
    data = json.load(f)

total = len(data)
correct = sum(
    1 for v in data.values()
    if v.get("is_correct_fixed", False) or v.get("is_correct", False)
)
acc = 100 * correct / total if total else 0

print("Total:", total)
print("Correct:", correct)
print("Accuracy:", f"{acc:.3f}%")
PY
```

Expected output: total 1514, correct 1069, accuracy 70.608%. The initial Tele-it RAG result showing 0% is invalid. The issue was mismatched answer formatting. The corrected field is `is_correct_fixed`.

29. Run Tele-it TeleQnA Without RAG

Script: `run_tele_it_teleqna_no_rag.py`.

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate

rm -f outputs/teleqna/tele_it_teleqna_no_rag.json
rm -f teleit_teleqna_no_rag.log

nohup python run_tele_it_teleqna_no_rag.py \
  > teleit_teleqna_no_rag.log 2>&1 &
```

```
echo "PID: $!"
```

Compute accuracy:

```
python - <<'PY'
import json

path = "outputs/teleqna/tele_it_teleqna_no_rag.json"

with open(path, "r", encoding="utf-8") as f:
    data = json.load(f)

total = len(data)
correct = sum(1 for v in data.values() if v.get("is_correct", False))
acc = 100 * correct / total if total else 0

print("Total:", total)
print("Correct:", correct)
print("Accuracy:", f"{acc:.3f}%")
PY
```

Expected: 790 / 1514 and 52.180%.

30. Run Tele-it TeleQnA No-RAG Paper-Faithful Variant

Script: `run_tele_it_teleqna_no_rag_paperfaithful.py`. Output: `outputs/teleqna/tele_it_teleqna_no_rag_paperfaithful.json`. Expected: 777 / 1514 and 51.321%. This prompt is closer to the paper's MCQ prompt wording but scored slightly below the simpler no-RAG prompt.

31. Run Chat3GPP Tele-Eval Generation

Script: `run_teleeval_eval_clean_rel1718_final.py`. Dataset: `data/teleeval_58627_exact.json`. Index: `index_clean_rel1718_final`. Output: `outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json`.

```
cd /scratch/work/yash22/Chat3GPP_fresh
source venv/bin/activate

rm -f outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json
rm -f teleeval_clean_rel1718_final.log

nohup python run_teleeval_eval_clean_rel1718_final.py \
  > teleeval_clean_rel1718_final.log 2>&1 &

echo "PID: $!"
```

Check progress:

```
tail -f teleeval_clean_rel1718_final.log
```

Check saved answers:

```
python - <<'PY'
import json, os

p = "outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json"
print("Exists:", os.path.exists(p))

if os.path.exists(p):
    with open(p, "r", encoding="utf-8") as f:
        d = json.load(f)
    print("Saved answers:", len(d))
PY
```

Expected final count: 58627.

32. Run Tele-it Tele-Eval With RAG

Script: `run_tele_it_teleeval_clean_rel1718_final.py`. Output: `outputs/teleeval/tele_it_teleeval_results_clean_rel1718_final.json`. This file is RAG-based because it uses `INDEX_NAME`, `search_rrf`, and retrieved context. Equivalent RAG-based Tele-it Tele-Eval variants may include `outputs/teleeval/tele_it_teleeval_results_cleanindex.json`, `outputs/teleeval/tele_it_teleeval_results_clean_rel1718_final.json`, and `outputs/teleeval/tele_it_teleeval_results_clean_rel1718_final_v2.json`. These should not be confused with no-RAG Tele-it.

33. Run Tele-it Tele-Eval Without RAG

Script: `run_teleit_teleeval_no_rag.py`. The final judged file is `outputs/teleeval/tele_it_no_rag_mixtral_eval_paperfaithful_v3.json`. This represents the no-RAG Tele-it Tele-Eval baseline after Mixtral judging.

34. Run Mixtral Judging

Final judge script: `run_mixtral_teleeval_judge_paperfaithful_v3.py`.

34.1 Judge Chat3GPP Tele-Eval

```
python run_mixtral_teleeval_judge_paperfaithful_v3.py \
  --input outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json \
  --output outputs/teleeval/chat3gpp_mixtral_eval_paperfaithful_v3.json \
  --model models/mixtral_q4/mixtral-8x7b-instruct-v0.1-q4_0.gguf \
  --n-gpu-layers 30 \
  --n-ctx 1024 \
  --n-threads 8 \
  --checkpoint-every 100
```

Expected: 72.736%.

34.2 Judge Tele-it No-RAG

```
python run_mixtral_teleeval_judge_paperfaithful_v3.py \
  --input outputs/teleeval/tele_it_teleeval_no_rag.json \
  --output outputs/teleeval/tele_it_no_rag_mixtral_eval_paperfaithful_v3.json \
  --model models/mixtral_q4/mixtral-8x7b-instruct-v0.1-q4_0.gguf \
  --n-gpu-layers 30 \
  --n-ctx 1024 \
  --n-threads 8 \
  --checkpoint-every 100
```

Expected: 25.273%.

34.3 Judge Tele-it With RAG

```
python run_mixtral_teleeval_judge_paperfaithful_v3.py \
  --input ablation/teleeval/tele_it_teleeval_results_clean_rel1718_final.json \
  --output ablation/teleeval/tele_it_rag_mixtral_eval_paperfaithful_v3.json \
  --model models/mixtral_q4/mixtral-8x7b-instruct-v0.1-q4_0.gguf \
  --n-gpu-layers 30 \
  --n-ctx 1024 \
  --n-threads 8 \
  --checkpoint-every 100
```

Expected: 87.489%. If the final judged Tele-it RAG output is stored under `outputs/teleeval/`, use `outputs/teleeval/tele_it_rag_mixtral_eval_paperfaithful_v3.json`.

35. Monitor Progress

Useful commands:

```
tail -f teleqna_clean_rel1718_final.log
tail -f teleit_teleqna_clean_rel1718_final.log
tail -f teleit_teleqna_no_rag.log
tail -f teleeval_clean_rel1718_final.log
```

Check JSON length:

```
python - <<'PY'
import json, os

files = [
    "outputs/teleqna/teleqna_results_clean_rel1718_final.json",
    "outputs/teleqna/chat3gpp_teleqna_clean_rel1718_final_77939.json",
    "outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final_fixed.json",
    "outputs/teleeval/chat3gpp_teleeval_results_clean_rel1718_final.json",
]

for p in files:
    print("=" * 80)
    print(p)
    print("Exists:", os.path.exists(p))
    if os.path.exists(p):
        with open(p, "r", encoding="utf-8") as f:
            d = json.load(f)
        print("Count:", len(d))
PY
```

36. Compute All TeleQnA Accuracies

```
python - <<'PY'
import json

files = [
    (
        "Chat3GPP TeleQnA with RAG",
        "outputs/teleqna/chat3gpp_teleqna_clean_rel1718_final_77939.json"
    ),
    (
        "Tele-it TeleQnA with RAG",
        "outputs/teleqna/tele_it_teleqna_results_clean_rel1718_final_fixed.json"
    ),
    (
        "Tele-it TeleQnA without RAG",
        "outputs/teleqna/tele_it_teleqna_no_rag.json"
    ),
    (
        "Tele-it TeleQnA no-RAG paper-style prompt",
        "outputs/teleqna/tele_it_teleqna_no_rag_paperfaithful.json"
    ),
]

for name, path in files:
    with open(path, "r", encoding="utf-8") as f:
        data = json.load(f)

    total = len(data)
    correct = sum(
        1 for v in data.values()
        if v.get("is_correct_fixed", False) or v.get("is_correct", False)
    )
    acc = 100 * correct / total if total else 0

    print("=" * 80)
    print(name)
    print("File:", path)
    print("Total:", total)
    print("Correct:", correct)
    print("Accuracy:", f"{acc:.3f}%")
PY
```

Expected final values: Chat3GPP TeleQnA with RAG: 1180 / 1514, 77.939%; Tele-it TeleQnA with RAG: 1069 / 1514, 70.608%; Tele-it TeleQnA without RAG: 790 / 1514, 52.180%; Tele-it TeleQnA no-RAG paper-style prompt: 777 / 1514, 51.321%.

37. Resume Behavior

Generation scripts load existing JSON files and skip completed question IDs. Therefore:

1. Rerunning a script resumes an interrupted job.
2. Do not delete partial JSON output unless intentionally restarting.
3. Use a new output filename for a new experiment.
4. Verify output counts before judging.
5. Do not judge incomplete Tele-Eval outputs.

Expected completed counts: TeleQnA: 1514; Tele-Eval: 58627.

38. Main Research Findings

38.1 MCQ Finding

The best MCQ result was achieved by Llama-3-8B-Instruct + RAG with 77.939% accuracy. Tele-it with RAG was implemented as an additional local extension beyond the original paper's reported Tele-it baseline; it achieved 70.608%, while Tele-it without RAG achieved 52.180% and Tele-it no-RAG paper-faithful prompt achieved 51.321%.

38.2 Open-Ended Finding

The best local open-ended result was achieved by Llama-3-8B-Tele-it + RAG with 87.489% accuracy. Chat3GPP with Llama-3-8B-Instruct + RAG achieved 72.736%, and Tele-it without RAG achieved 25.273%.

38.3 Main Insight

RAG is essential for telecom QA. Fine-tuning alone does not provide sufficient performance on standards-specific questions. The best model depends on the task: Llama-3-8B-Instruct + RAG is best for MCQ, while Llama-3-8B-Tele-it + RAG is best for open-ended generation.

39. Research Gap

Existing telecom LLMs such as Llama-3-8B-Tele-it are trained on telecom-domain data, but they do not fully memorize or retrieve all relevant standards information from model weights alone. This creates the research gap: how can domain-adapted LLMs and retrieval systems be combined optimally for different telecom QA formats?

The results show:

- Tele-it no-RAG MCQ: 51.321%
- Tele-it + RAG MCQ: 70.608%
- Tele-it no-RAG Tele-Eval: 25.273%
- Tele-it + RAG Tele-Eval: 87.489%

This proves that retrieval substantially improves the Tele-it model, especially for open-ended questions. At the same time, Llama-3-8B-Instruct + RAG remains better for MCQs, suggesting that the best model depends on whether the task is option selection or free-form generation.

40. Contributions

This project contributes:

1. A reproduced Chat3GPP-style RAG pipeline.
2. A Release 17/18 filtered 3GPP knowledge base.
3. A final frozen Elasticsearch index for 3GPP QA.
4. A verified TeleQnA subset matching the paper.
5. A verified Tele-Eval subset matching the paper.
6. A comparison of Llama-3-8B-Instruct and Llama-3-8B-Tele-it.
7. A comparison of RAG and no-RAG settings.
8. A Tele-it with RAG ablation study, added as an extension beyond the original paper's reported Tele-it baseline.
9. A clear paper-versus-local comparison explaining why the local 77.939% Chat3GPP result is close to but slightly different from the paper's 78.7% result.
10. Local Tele-Eval judging using quantized Mixtral.
11. Evidence that Tele-it benefits dramatically from retrieval.
12. Evidence that Llama-3-8B-Instruct + RAG is strongest on MCQs.
13. A reproducible execution workflow for indexing, generation, and judging.
14. A preserved frozen final implementation for future extension.

41. Known Implementation Limitations

The main limitations are:

1. Tables and figures are excluded.
2. The local corpus used processed text rather than the exact raw `.doc/.docx` paper pipeline.
3. Local reproduced accuracies are close to the paper but not bit-identical because raw 3GPP Word documents were not used for final indexing.
4. Retrieval recall is not measured separately.
5. No complete BM25-only versus dense-only ablation exists.
6. Dense retrieval uses scripted cosine rather than explicit HNSW in final local code.
7. Zero-overlap chunks may split evidence across chunk boundaries.
8. Quantized Mixtral has not been calibrated against full-precision Mixtral.
9. Judge context is limited to 1024 tokens.
10. Statistical significance tests are absent.
11. Release is not used as an explicit Elasticsearch filter during retrieval.
12. RRF and K values have not been systematically optimized.
13. Generated answers do not expose clause citations.
14. Temporal update testing is missing.
15. MCQ selection bias remains possible.
16. Multistep protocol reasoning requires separate evaluation.
17. Tele-it's MCQ/open-ended behavior needs controlled investigation.
18. Results may differ from the paper because of hardware, package versions, corpus preprocessing, generation settings, and local judge quantization.
19. Some experimental folders contain non-final scripts and should not be used blindly.
20. No preserved Elasticsearch snapshot is mentioned.
21. Tables, diagrams, and message-flow figures are not incorporated into retrieval.

42. Future Work

Future work should include:

1. Running the exact raw `.doc/.docx` preprocessing pipeline.
2. Using the full unquantized Mixtral evaluator.
3. Testing additional telecom LLMs.
4. Combining Tele-it with improved retrieval and reranking.
5. Studying why Tele-it + RAG performs best on open-ended tasks but not MCQs.
6. Evaluating alternative rerankers.
7. Performing error analysis by 3GPP series.
8. Adding multimodal support for tables and figures in 3GPP documents.
9. Reproducing Telco-RAG under identical conditions.
10. Building a stable Dockerized reproduction environment.
11. Adding release-aware retrieval filtering.
12. Adding clause-level citations in generated answers.
13. Measuring retrieval recall separately from answer generation.
14. Testing BM25-only, dense-only, and reranker-only ablations.
15. Testing overlapping chunks.
16. Testing multi-hop retrieval.
17. Testing graph-based retrieval.
18. Testing real-time 3GPP release updates.
19. Evaluating larger generator models.
20. Studying controlled prompt effects for Tele-it.

43. Chat3GPP Result Interpretation

The frozen Chat3GPP system achieved 77.939% accuracy, with 1180 / 1514 correct, on the complete 1514-question Release 17/18 TeleQnA subset using a general Llama-3-8B-Instruct generator without telecom-specific fine-tuning. The original paper reports 78.7% for Chat3GPP on TeleQnA; the local result is lower by only 0.761 percentage points, which is expected because the original paper indexed raw 3GPP Word documents while the local implementation indexed processed and cleaned text content. It exceeded paper-faithful Llama-3-8B-Tele-it without RAG by 26.618 percentage points and 403 additional correct answers. It also exceeded Tele-it using the same RAG pipeline by 7.331 percentage points.

The improvement resulted primarily from hierarchical corpus cleaning, Release 17/18 restriction, conservative normalization, hybrid BM25 and dense retrieval, Reciprocal Rank Fusion, BGE-M3 ColBERT-style reranking, deterministic generation, and corrected MCQ answer parsing. For open-ended Tele-Eval, Tele-it and retrieval were complementary. The strongest local Q4 Mixtral-judge score was Tele-it + RAG at 87.489%.

The final conclusion is that retrieval and corpus quality are more valuable than telecom fine-tuning alone for exact 3GPP MCQ answering. Telecom fine-tuning becomes highly beneficial for open-ended answer generation when combined with strong retrieved evidence. Therefore, future telecom QA systems should combine domain-adapted LLMs with high-quality retrieval over current standards documents.

44. Chat3GPP-2.0 Overview

Chat3GPP-2.0 combines:

- A Neo4j telecommunications knowledge graph.
- Graph-guided document selection.
- Streaming document ingestion through Pathway.
- Document parsing and OCR.
- Chunking and hybrid retrieval.
- Reranking.
- Context-grounded answer generation.
- Separate support for global 3GPP documents and private user uploads.

The current implementation depends on paid Voyage embedding/reranking services and API-based generation models. At large corpus and query volumes, these services could cost tens of thousands of rupees. A fully local alternative can instead use Llama-3, BGE-M3, and Elasticsearch.

45. Telecom-Data Ingestion into Neo4j

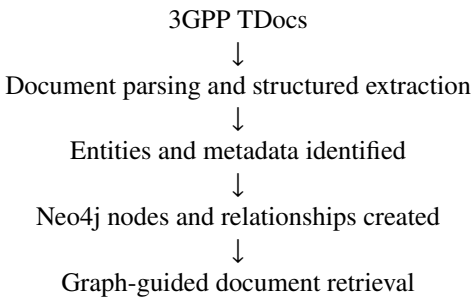
The 3GPP telecom/TDoc dataset is processed and ingested into a Neo4j knowledge graph.
The graph contains nodes such as:

- `Document`
- `TechnologyEntity`
- `WorkingGroup`
- `Meeting`
- `Agenda`
- `Contributor`

Relationships represent information such as:

- A contributor submitting a document.
- A document belonging to a meeting.
- A document appearing under an agenda item.
- A working group managing a document.
- A document mentioning a technical entity.
- Multiple documents discussing related technologies.

This converts the flat TDoc collection into a structured telecommunications knowledge graph.



46. Graph-Guided Retrieval

When a question is received, Chat3GPP-2.0 first searches Neo4j. The graph retrieval uses signals such as direct document-content matches, technology-entity matches, meeting and agenda relationships, working-group associations, contributor relationships, and documents connected through shared entities. Approximately 15 relevant documents are selected before passage-level retrieval begins. This prevents the RAG system from searching every chunk from all 10356 documents for every query.

47. Live Document Ingestion

Documents selected by Neo4j are retrieved using their stored 3GPP source paths. If a selected document is not already available locally:

1. Its archive is downloaded.
2. The required document is extracted.
3. It is placed in `temp_rag_space`.
4. The streaming ingestion service detects it.
5. It is parsed, chunked, embedded, and indexed.

Documents remain in `temp_rag_space`, creating an accumulating warm cache. Frequently requested documents therefore need to be downloaded and processed only once.

48. Current Pathway Processing

Pathway monitors two document spaces and supporting services.

Table 12: Chat3GPP-2.0 service layout.

Service	Port	Purpose
Global document store	4004	Cached 3GPP and globally available documents
User document store	4006	Private user-uploaded documents
RAG orchestration server	4005	Retrieval, reranking, and generation
Upload server	8000	Authenticated file upload management

When a new document appears, Pathway automatically performs file detection, unstructured parsing, OCR where required, recursive text splitting, embedding, lexical and vector indexing, and persistent checkpointing. The current chunking configuration is 4000 characters with an 800-character overlap.

49. Current Paid Configuration

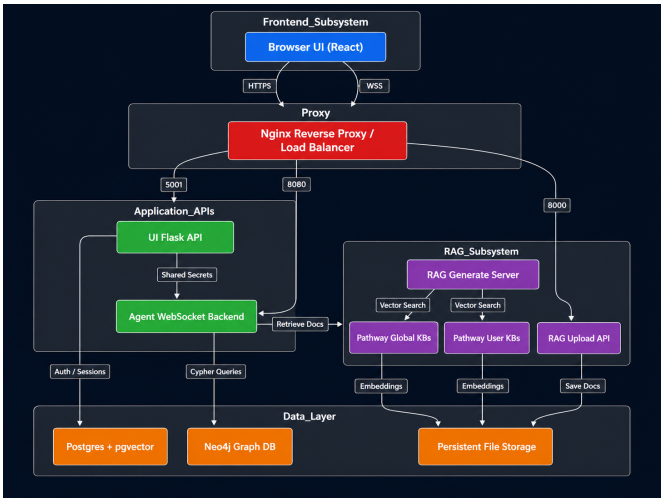


Figure 2: Chat3GPP-2.0 architecture.

Table 13: Current paid Chat3GPP-2.0 configuration.

Component	Current Configuration
Embedding model	Voyage `voyage-3-large`
Vector retrieval	USearch KNN
Lexical retrieval	Tantivy BM25
Fusion	Reciprocal Rank Fusion
Reranker	Voyage `rerank-2`
Generator	DeepSeek/OpenAI-compatible API models
Streaming store	Pathway DocumentStore
Knowledge graph	Neo4j

Embedding thousands of documents and reranking large numbers of queries through paid APIs can create substantial recurring costs. Depending on document volume and usage, this may reach tens of thousands of rupees.

50. Proposed Free Local Alternative

Table 14: Proposed free local configuration.

Parameter	Proposed Local Setting
LLM	Llama-3-8B-Instruct
Open-ended LLM option	Llama-3-8B-Tele-it
Embedding model	BGE-M3
Rerank model	BGE-M3
Embedding database	Elasticsearch
Indexing strategy	Inverted index + HNSW
Chunk size	Approximately 1250 characters
Splitter	RecursiveCharacterTextSplitter
Top-K1	1000
Top-K2	5

These models and components can run locally, eliminating per-document embedding, per-query reranking, and generation API charges. Hardware, electricity, and server-maintenance costs still remain.

51. Proposed Alternative Retrieval Pipeline

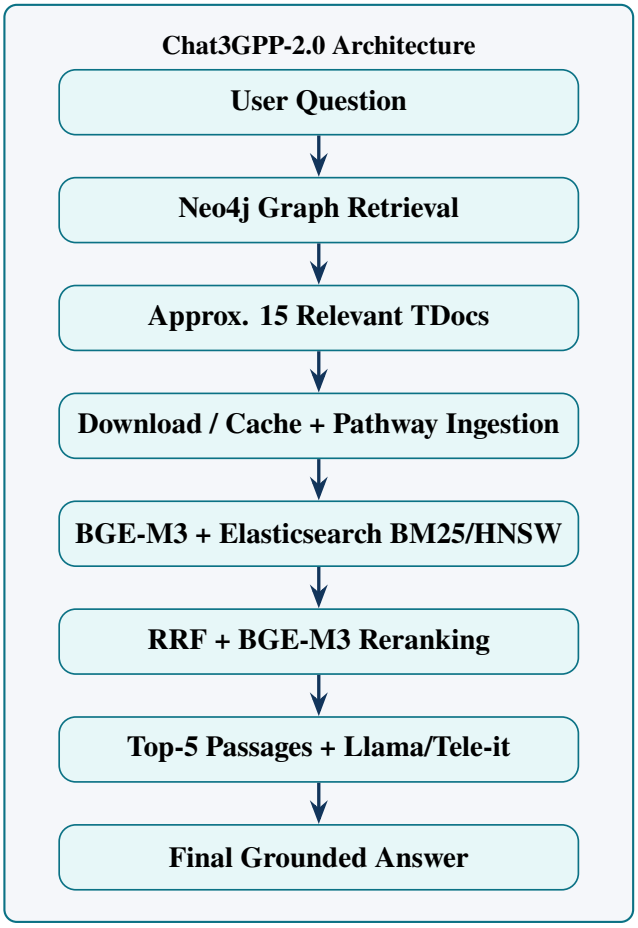


Figure 3: Proposed free architecture for Chat3GPP-2.0.

The proposed local Chat3GPP-2.0 retrieval pipeline is:

User question
↓
Neo4j graph retrieval
↓
Approximately 15 relevant TDocs selected
↓
Documents downloaded or loaded from cache
↓
Text extraction and cleaning
↓
Recursive 1250-character chunking
↓
BGE-M3 dense embeddings
↓
Elasticsearch indexing
↓
Inverted text index + HNSW dense-vector index
↓
BM25 Top-1000 retrieval + BGE-M3 HNSW Top-1000 retrieval
↓
Reciprocal Rank Fusion
↓
BGE-M3 reranking
↓
Top five passages
↓
Llama-3-8B-Instruct or Tele-it
↓
Final grounded answer

52. BGE-M3 Embeddings in Chat3GPP-2.0

BGE-M3 can generate 1024-dimensional dense embeddings locally. During indexing:

```
Document chunk -> BGE-M3 -> 1024-dimensional vector
```

During querying:

```
Question -> BGE-M3 -> query vector
```

The query and document vectors are compared using cosine similarity. Advantages include no paid embedding API, strong multilingual and technical semantic representation, local processing of potentially confidential documents, reusable embeddings, and support for dense and ColBERT representations.

53. Elasticsearch Inverted Index

The chunk content is stored as searchable text in Elasticsearch. The inverted index conceptually stores:

```
Term -> documents/chunks containing the term
```

For example:

```
PDCCH       -> chunks 20, 91 and 183
TS 38.214   -> chunks 17, 42 and 206
R1-2408765  -> chunks 37 and 122
```

BM25 ranks chunks using query-term occurrence, term rarity, document length, and repeated term frequency. This is particularly useful for exact 3GPP identifiers that semantic embeddings may not represent precisely.

54. HNSW Vector Index

BGE-M3 vectors can be stored in an Elasticsearch HNSW index. HNSW organizes vectors into a multilayer proximity graph:

- Every chunk becomes a vector node.
- Semantically close vectors are connected.
- Upper layers support rapid movement across the index.
- Lower layers perform finer local searches.
- Only a subset of all vectors must be examined.

This provides efficient approximate-nearest-neighbour retrieval without comparing the query with every indexed chunk. The dense branch returns the top 1000 candidates.

55. Hybrid Retrieval and RRF

Two candidate lists are produced: BM25 Top-1000 and BGE-M3/HNSW Top-1000. Reciprocal Rank Fusion combines them:

```
RRF score =
    1 / (constant + BM25 rank)
    +
    1 / (constant + dense rank)
```

This combination is effective because BM25 provides exact technical matching, dense retrieval provides semantic matching, RRF prevents either retriever from completely controlling the result, and passages found by both systems receive stronger combined ranks.

56. BGE-M3 Reranking

After RRF, BGE-M3 can rerank the fused candidates using ColBERT late interaction. The reranker compares individual query tokens with individual passage tokens. This is valuable for technical phrases such as:

- `PDCCH monitoring occasion`
- `Nsmf_PDUSession_UpdateSMContext`
- `TS 38.214`
- `N2 handover`
- `MCS table index`

The reranking stage produces a more precise final ordering, from which the top five passages are selected. A dedicated local alternative is `BAAI/bge-reranker-v2-m3`, but the requested configuration can use BGE-M3 for both dense embeddings and ColBERT reranking.

57. Local Generation Models

57.1 MCQ Questions

Use Llama-3-8B-Instruct. The model receives five retrieved passages, the question, all answer options, and a strict output-format instruction. Options should be added after retrieval so their wording does not influence document search.

57.2 Open-Ended Questions

Use Llama-3-8B-Tele-it. Tele-it's telecommunications adaptation is useful for producing detailed open-ended answers once accurate retrieved evidence is supplied. These models can be hosted locally and do not require OpenAI API keys.

58. Recommended Final Chat3GPP-2.0 Configuration

Table 15: Recommended final Chat3GPP-2.0 local configuration.

Component	Recommended Setting
Knowledge graph	Neo4j
Graph retrieval	Approximately 15 relevant TDocs
Document ingestion	Graph-guided download and persistent warm cache
Chunking	RecursiveCharacterTextSplitter, approx. 1250 characters
Embedding	BAAI/bge-m3, 1024-dimensional vectors
Text index	Elasticsearch inverted index
Vector index	Elasticsearch HNSW
Lexical retrieval	BM25 Top-1000
Dense retrieval	BGE-M3/HNSW Top-1000
Fusion	Reciprocal Rank Fusion
Reranking	BGE-M3 ColBERT late interaction
Final context	Top five passages
MCQ model	Llama-3-8B-Instruct
Open-ended model	Llama-3-8B-Tele-it

59. Chat3GPP-2.0 Result Interpretation

Chat3GPP-2.0 first ingests structured 3GPP telecom data into Neo4j and uses graph relationships to select a focused group of relevant documents. These documents are then parsed, chunked, and searched through the passage-level RAG pipeline.

The current Voyage and API-based model configuration can produce substantial recurring costs. It can be replaced with BGE-M3 embeddings, Elasticsearch inverted and HNSW indexes, BGE-M3 reranking, Llama-3-8B-Instruct for MCQs, and Llama-3-8B-Tele-it for open-ended questions. This retains graph-guided, hybrid, and reranked retrieval while removing paid embedding, reranking, and generation API dependencies.

60. Conclusion

The internship work covered two connected systems:

1. Chat3GPP, a reproduced and extended RAG pipeline for 3GPP Release 17/18 standards-based question answering.
2. Chat3GPP-2.0, a graph-guided RAG architecture that uses Neo4j and streaming ingestion to select and process relevant 3GPP documents dynamically.

The Chat3GPP results show that corpus engineering, release filtering, hybrid retrieval, RRF fusion, and reranking are critical for exact telecom QA. The best MCQ result was achieved by Llama-3-8B-Instruct with final RAG, while the best open-ended result was achieved by Llama-3-8B-Tele-it with final RAG.

The Chat3GPP-2.0 design extends this direction by adding graph-guided document selection and live ingestion. Its current paid components can be replaced with a fully local stack based on Neo4j, Elasticsearch, BGE-M3, Llama-3-8B-Instruct, and Llama-3-8B-Tele-it.

The combined conclusion is that future telecom question-answering systems should not rely only on model fine-tuning. They should combine domain-adapted LLMs with accurate, current, retrievable standards knowledge, strong indexing, hybrid retrieval, reranking, and controlled generation.

Bibliography

[1] L. Huang, M. Zhao, L. Xiao, X. Zhang, and J. Hu, "Chat3GPP: An Open-Source Retrieval-Augmented Generation Framework for 3GPP Documents," arXiv:2501.13954, 2025.

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems, 2020.

[3] A. Maatouk et al., "TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge," arXiv:2310.15051, 2023.

[4] A. Maatouk, K. C. Ampudia, R. Ying, and L. Tassiulas, "Tele-LLMs: A Series of Specialized Large Language Models for Telecommunications," arXiv:2409.05314, 2024.

[5] A.-L. Bornea, F. Ayed, A. D. Domenico, N. Piovesan, and A. Maatouk, "Telco-RAG: Navigating the Challenges of Retrieval-Augmented Language Models for Telecommunications," arXiv:2404.15939, 2024.

[6] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, 2009.

[7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," Proceedings of SIGIR, 2009.

[8] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, 2017.

[9] A. Q. Jiang et al., "Mixtral of Experts," arXiv:2401.04088, 2024.

[10] J. Chen et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," 2024.

[11] Neo4j, "Neo4j Graph Database Documentation." Available: https://neo4j.com/docs/

[12] Pathway, "Pathway Documentation." Available: https://pathway.com/docs/

[13] Elastic, "Elasticsearch Documentation." Available: https://www.elastic.co/guide/